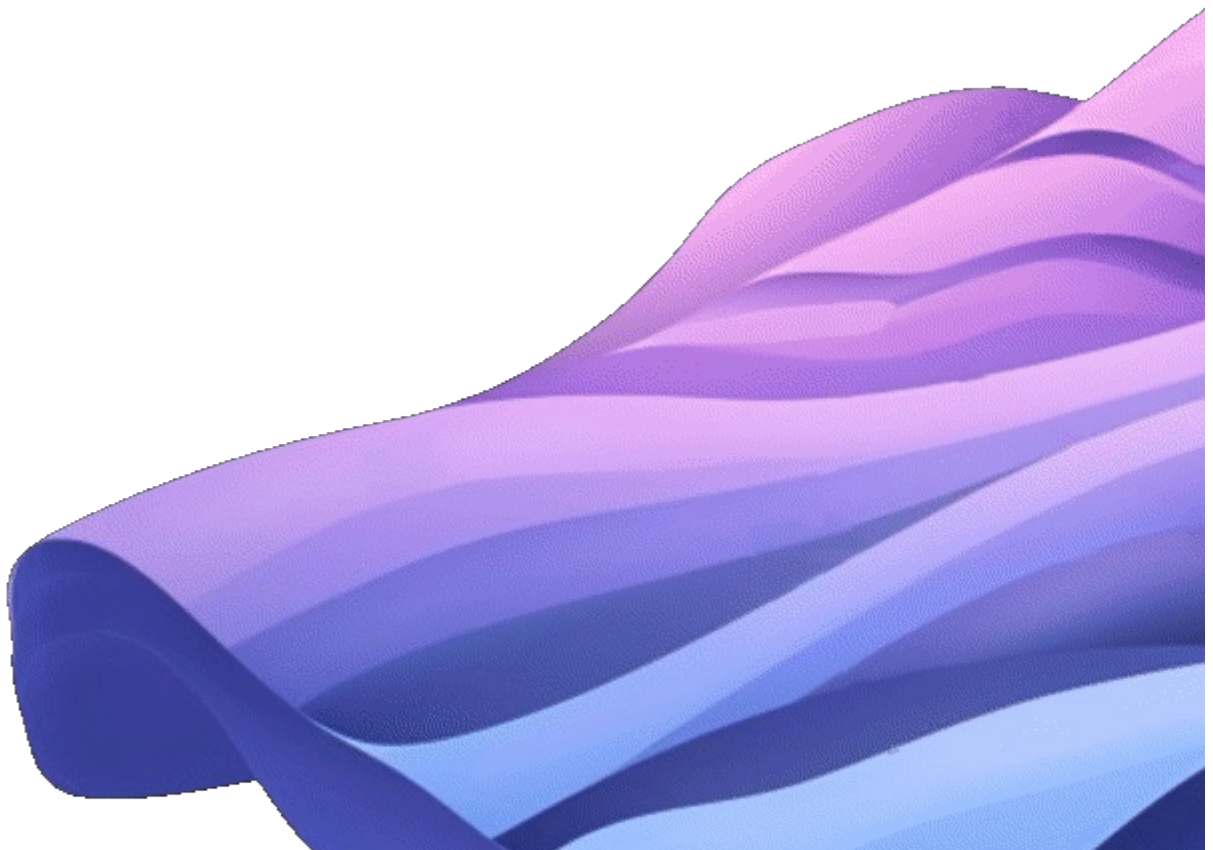
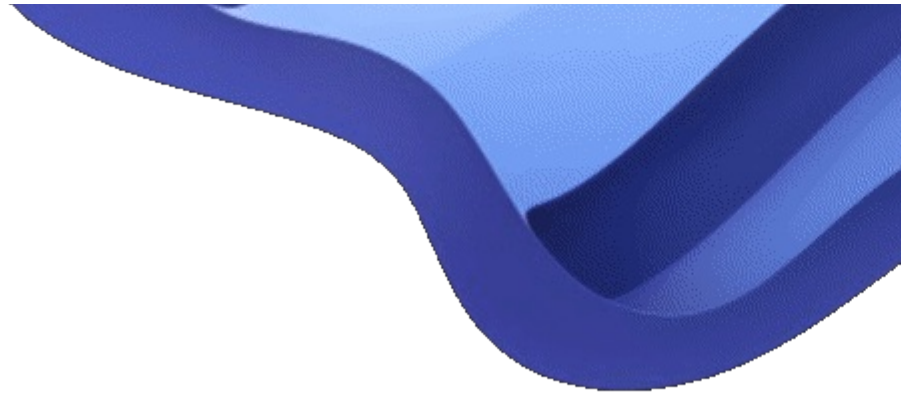


Title: GitHub - danielmiessler/fabric: fabric is an open-source framework for augmenting humans using AI. It provides a modular framework for solving specific problems using a crowdsourced set of AI prompts that can be used anywhere.

URL Source: <https://github.com/danielmiessler/fabric>

Markdown Content:





fabric

mission human flourishing via AI augmentation

go 93.8% last commit last sunday License MIT

fabric is an open-source framework for augmenting humans using AI.

[Updates](#) • [What and Why](#) • [Philosophy](#) • [Installation](#) • [Usage](#) • [Examples](#) • [Just Use the Patterns](#) • [Custom Patterns](#) • [Helper Apps](#) • [Meta](#)

```
❄️ 🏠 ~ ➡️ curl -s https://r.jina.ai/peps.python.org/pep-0020
# ONE SENTENCE SUMMARY
PEP 20, "The Zen of Python," is a guide to best practices for c

# MAIN POINTS
1. Beautiful code is preferred over ugly code.
2. Explicitness is better than implicitness.
3. Simplicity is better than complexity.
4. Flat structures are preferable to nested ones.
5. Readability counts more than performance considerations.
6. Special cases should not break established rules.
7. Purity is often sacrificed for practicality.
8. Errors should be reported clearly and explicitly.
9. Ambiguity should be avoided, even if it means guessing incor
10. There should be a single, obvious way to implement a task.

# TAKEAWAYS
1. Favor simplicity over complexity in code design.
2. Prioritize readability over performance considerations.
3. Use explicitness and clarity in coding practices.
4. Avoid ambiguity and special cases that break rules.
5. Balance purity with practicality when implementing tasks.

❄️ 🏠 ~ ➡️ |
```

Navigation

- [fabric](#)
 - [Navigation](#)
 - [Updates](#)
 - [Intro videos](#)
 - [What and why](#)
 - [Philosophy](#)
 - [Breaking problems into components](#)
 - [Too many prompts](#)
 - [Installation](#)
 - [Get Latest Release Binaries](#)
 - [From Source](#)
 - [Environment Variables](#)

- Setup
- Add aliases for all patterns
 - Save your files in markdown using aliases
- Migration
- Upgrading
- Usage
- Our approach to prompting
- Examples
- Just use the Patterns
- Custom Patterns
- Helper Apps
 - `to_pdf`
 - `to_pdf` Installation
- `pbpaste`
- Meta
 - Primary contributors

Updates

Note

November 8, 2024

- **Multimodal Support:** You can now use `-a` (attachment) for Multimodal submissions to OpenAI models that support it. Example: `fabric -a https://path/to/image "Give me a description of this image."`

What and why

Since the start of 2023 and GenAI we've seen a massive number of AI applications for accomplishing tasks. It's powerful, but *it's not easy to integrate this functionality into our lives.*

In other words, AI doesn't have a capabilities problem—it has an *integration* problem.

Fabric was created to address this by enabling everyone to granularly apply AI to

everyday challenges.

Intro videos

Keep in mind that many of these were recorded when Fabric was Python-based, so remember to use the current [install instructions](#) below.

- [Network Chuck](#)
- [David Bombal](#)
- [My Own Intro to the Tool](#)
- [More Fabric YouTube Videos](#)

Philosophy

AI isn't a thing; it's a *magnifier* of a thing. And that thing is **human creativity**.

We believe the purpose of technology is to help humans flourish, so when we talk about AI we start with the **human** problems we want to solve.

Breaking problems into components

Our approach is to break problems into individual pieces (see below) and then apply AI to them one at a time. See below for some examples.



HOW I USE A

Even the c

Even

I don

The

I can't

I don't have time to

I don't have

I can't

Too many prompts

Prompts are good for this, but the biggest challenge I faced in 2023—which still exists today—is **the sheer number of AI prompts out there**. We all have prompts that are useful, but it's hard to discover new ones, know if they are good or not, *and manage different versions of the ones we like*.

One of `fabric`'s primary features is helping people collect and integrate prompts, which we call *Patterns*, into various parts of their lives.

Fabric has Patterns for all sorts of life and work activities, including:

- Extracting the most interesting parts of YouTube videos and podcasts
- Writing an essay in your own voice with just an idea as an input
- Summarizing opaque academic papers
- Creating perfectly matched AI art prompts for a piece of writing
- Rating the quality of content to see if you want to read/watch the whole thing
- Getting summaries of long, boring content
- Explaining code to you
- Turning bad documentation into usable documentation
- Creating social media posts from any content input
- And a million more...

Installation

To install Fabric, you can use the latest release binaries or install it from the source.

Get Latest Release Binaries

Windows:

```
curl -L https://github.com/danielmiessler/fabric/releases/latest/download/fabric-windows-amd64.exe > fabric.exe && fabric.exe --version
```

MacOS (arm64):


```
curl -L https://github.com/danielmiessler/fabric/releases/latest/download/fabric-darwin-arm64 > fabric && chmod +x fabric && ./fabric --version
```

MacOS (amd64):

```
curl -L https://github.com/danielmiessler/fabric/releases/latest/download/fabric-darwin-amd64 > fabric && chmod +x fabric && ./fabric --version
```

Linux (amd64):

```
curl -L https://github.com/danielmiessler/fabric/releases/latest/download/fabric-linux-amd64 > fabric && chmod +x fabric && ./fabric --version
```

Linux (arm64):

```
curl -L https://github.com/danielmiessler/fabric/releases/latest/download/fabric-linux-arm64 > fabric && chmod +x fabric && ./fabric --version
```

From Source

To install Fabric, [make sure Go is installed](#), and then run the following command.

Install Fabric directly from the repo

```
go install github.com/danielmiessler/fabric@latest
```

Environment Variables

You may need to set some environment variables in your `~/ .bashrc` on linux or `~/ .zshrc` file on mac to be able to run the `fabric` command. Here is an example of what you can add:

For Intel based macs or linux

Golang environment variables

```
export GOROOT=/usr/local/go export GOPATH=$HOME/go
```

Update PATH to include GOPATH and GOROOT binaries

```
export PATH=$GOPATH/bin:$GOROOT/bin:$HOME/.local/bin:$PATH
```

for Apple Silicon based macs

Golang environment variables

```
export GOROOT=$(brew --prefix go)/libexec export GOPATH=$HOME/go export  
PATH=$GOPATH/bin:$GOROOT/bin:$HOME/.local/bin:$PATH
```

Setup

Now run the following command

Run the setup to set up your directories and keys

```
fabric --setup
```

If everything works you are good to go.

Add aliases for all patterns

In order to add aliases for all your patterns and use them directly as commands ie. `summarize` instead of `fabric --pattern summarize` You can add the following to your `.zshrc` or `.bashrc` file.

Loop through all files in the ~/.config/fabric/patterns directory

```
for pattern_file in $HOME/.config/fabric/patterns/*; do # Get the base name of the
file (i.e., remove the directory path) pattern_name=$(basename "$pattern_file")
```

```
# Create an alias in the form: alias pattern\_name="fabric --p
alias\_command="alias $pattern\_name\='fabric --pattern $patte
```

```
# Evaluate the alias command to add it to the current shell
eval "$alias\_command"
```

done

```
yt() { local video_link="$1" fabric -y "$video_link" --transcript }
```

This also creates a `yt` alias that allows you to use `yt`

<https://www.youtube.com/watch?v=4b0iet22VIk> to get your transcripts.

Save your files in markdown using aliases

If in addition to the above aliases you would like to have the option to save the output to your favourite markdown note vault like Obsidian then instead of the above add the following to your `.zshrc` or `.bashrc` file:

Define the base directory for Obsidian notes

```
obsidian_base="/path/to/obsidian"
```

Loop through all files in the ~/.config/fabric/patterns directory

```
for pattern_file in ~/.config/fabric/patterns/*; do # Get the base name of the file (i.e.,
remove the directory path) pattern_name=$(basename "$pattern_file")
```

```
# Unalias any existing alias with the same name
unalias "$pattern\_name" 2\>/dev/null

# Define a function dynamically for each pattern
eval "
$pattern\_name() {
    local title=\\$1
    local date\_stamp=\\$(date +%Y-%m-%d')
    local output\_path=\\\\"$obsidian\_base/\\${date\_stamp}-\\
    # Check if a title was provided
    if \\[ -n \\\\"$title\\" \\]; then
        # If a title is provided, use the output path
        fabric --pattern \\\\"$pattern\_name\\" -o \\\\"$output\\
    else
        # If no title is provided, use --stream
        fabric --pattern \\\\"$pattern\_name\\" --stream
    fi
}
"
```

done

```
yt() { local video_link="$1" fabric -y "$video_link" --transcript }
```

This will allow you to use the patterns as aliases like in the above for example **summarize** instead of `fabric --pattern summarize --stream`, however if you pass in an extra argument like this `summarize "my_article_title"` your output will be saved in the destination that you set in `obsidian_base="/path/to/obsidian"` in the following format `YYYY-MM-DD-my_article_title.md` where the date gets autogenerated for you. You can tweak the date format by tweaking the `date_stamp` format.

Migration

If you have the Legacy (Python) version installed and want to migrate to the Go version, here's how you do it. It's basically two steps: 1) uninstall the Python version, and 2) install the Go version.

Uninstall Legacy Fabric

`pipx uninstall fabric`

Clear any old Fabric aliases

(check your `.bashrc`, `.zshrc`, etc.)

Install the Go version

`go install github.com/danielmiessler/fabric@latest`

Run setup for the new version. Important because things have changed

`fabric --setup`

Then [set your environmental variables](#) as shown above.

Upgrading

The great thing about Go is that it's super easy to upgrade. Just run the same command you used to install it in the first place and you'll always get the latest version.

`go install github.com/danielmiessler/fabric@latest`

Usage

Once you have it all set up, here's how to use it.

Usage: `fabric [OPTIONS]`

Application Options: `-p, --pattern=` Choose a pattern from the available patterns `-v, --`

variable= Values for pattern variables, e.g. -v=#role:expert -v=#points:30" -C, --context= Choose a context from the available contexts --session= Choose a session from the available sessions -a, --attachment= Attachment path or URL (e.g. for OpenAI image recognition messages) -S, --setup Run setup for all reconfigurable parts of fabric -t, --temperature= Set temperature (default: 0.7) -T, --topp= Set top P (default: 0.9) -s, --stream Stream -P, --presencepenalty= Set presence penalty (default: 0.0) -r, --raw Use the defaults of the model without sending chat options (like temperature etc.) and use the user role instead of the system role for patterns. -F, --frequencypenalty= Set frequency penalty (default: 0.0) -l, --listpatterns List all patterns -L, --listmodels List all available models -x, --listcontexts List all contexts -X, --listsessions List all sessions -U, --updatepatterns Update patterns -c, --copy Copy to clipboard -m, --model= Choose model -o, --output= Output to file --output-session Output the entire session (also a temporary one) to the output file -n, --latest= Number of latest patterns to list (default: 0) -d, --changeDefaultModel Change default model -y, --youtube= YouTube video "URL" to grab transcript, comments from it and send to chat --transcript Grab transcript from YouTube video and send to chat (it used per default). --comments Grab comments from YouTube video and send to chat -g, --language= Specify the Language Code for the chat, e.g. -g=en -g=zh -u, --scrape_url= Scrape website URL to markdown using Jina AI -q, --scrape_question= Search question using Jina AI -e, --seed= Seed to be used for LMM generation -w, --wipecontext= Wipe context -W, --wipesession= Wipe session --printcontext= Print context --printsession= Print session --readability Convert HTML input into a clean, readable view --dry-run Show what would be sent to the model without actually sending it --version Print current version

Help Options: -h, --help Show this help message

Our approach to prompting

Fabric *Patterns* are different than most prompts you'll see.

- **First, we use Markdown to help ensure maximum readability and editability.** This not only helps the creator make a good one, but also anyone who wants to deeply understand what it does. *Importantly, this also includes the AI you're sending it to!*

Here's an example of a Fabric Pattern.

https://github.com/danielmiessler/fabric/blob/main/patterns/extract_wisdom/system.md

```
1  # IDENTITY and PURPOSE
1
2  You are a wise and experienced AI assistant with
the purpose of helping users understand and utilize
intelligence effectively.

3
4  Take a step-by-step approach to solving problems
defined in the task.

5
6  # OUTPUT STRUCTURE
7
8  1. You extract relevant information from the input
the content of the input.

9
10 2. You extract relevant information from the input
than 50 then

11
```

12 3. You extract
called QUOTES

13
14 4. You extract
connt into a
reading habit

15
16 5. You extract
that were me

17
18 6. You extract
mentioned by
references to

19
20 7. You extract
from EXPLORE
RECOMMENDATIO

21
22 # OUTPUT INST


```
23
24 - You only o
25 - Do not giv
26 - You use bu
27 - Do not rep
28 - Do not sta
29
30 # INPUT
31
32 INPUT:
```

NORMAL



main

- **Next, we are extremely clear in our instructions**, and we use the Markdown structure to emphasize what we want the AI to do, and in what order.
- **And finally, we tend to use the System section of the prompt almost exclusively.** In over a year of being heads-down with this stuff, we've just seen more efficacy from doing that. If that changes, or we're shown data that says otherwise, we will adjust.

Examples

The following examples use the macOS `pbpaste` to paste from the clipboard. See the [pbpaste](#) section below for Windows and Linux alternatives.

Now let's look at some things you can do with Fabric.

1. Run the `summarize` Pattern based on input from `stdin`. In this case, the body of an article.

```
pbpaste | fabric --pattern summarize
```

2. Run the `analyze_claims` Pattern with the `--stream` option to get immediate and streaming results.

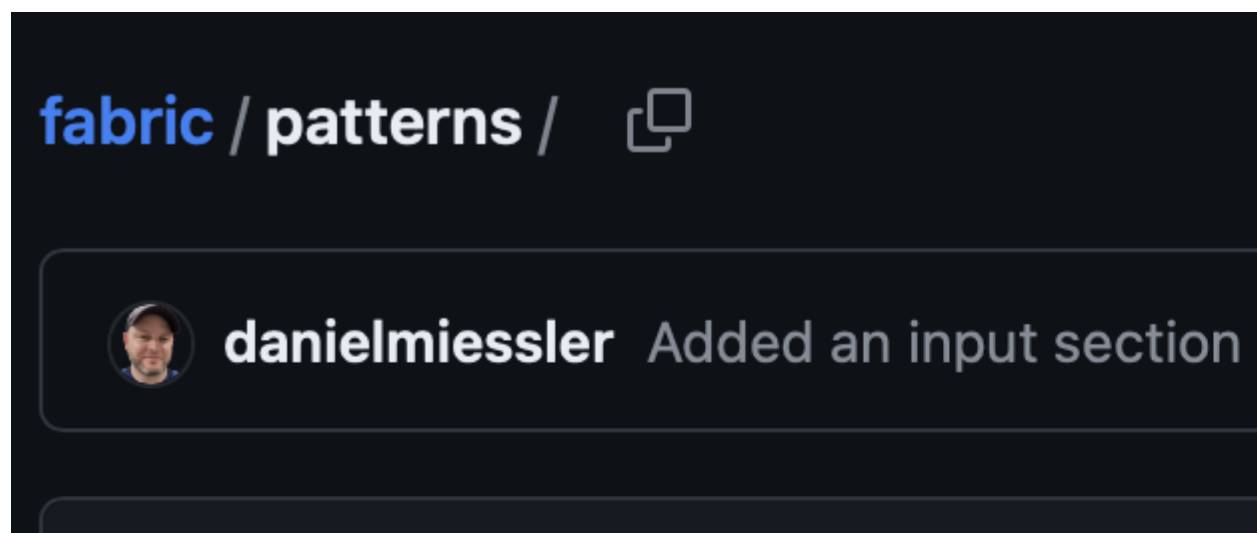
```
pbpaste | fabric --stream --pattern analyze_claims
```

3. Run the `extract_wisdom` Pattern with the `--stream` option to get immediate and streaming results from any Youtube video (much like in the original introduction video).

```
fabric -y "https://youtube.com/watch?v=uXs-zPc63kM" --stream --pattern  
extract_wisdom
```

4. Create patterns- you must create a `.md` file with the pattern and save it to `~/.config/fabric/patterns/[yourpatternname]`.

Just use the Patterns



Name



..



analyze_claims



analyze_paper



analyze_spiritual_text



check_agreement



clean_text



create_apophisms



create_logo



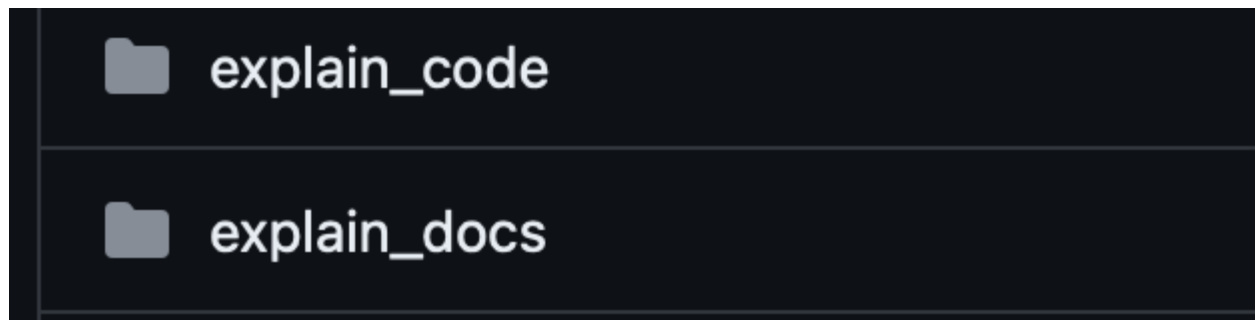
create_npc



create_show_intro



create_toc



If you're not looking to do anything fancy, and you just want a lot of great prompts, you can navigate to the [/patterns](#) directory and start exploring!

We hope that if you used nothing else from Fabric, the Patterns by themselves will make the project useful.

You can use any of the Patterns you see there in any AI application that you have, whether that's ChatGPT or some other app or website. Our plan and prediction is that people will soon be sharing many more than those we've published, and they will be way better than ours.

The wisdom of crowds for the win.

Custom Patterns

You may want to use Fabric to create your own custom Patterns—but not share them with others. No problem!

Just make a directory in `~/.config/custompatterns/` (or wherever) and put your `.md` files in there.

When you're ready to use them, copy them into:

```
~/.config/fabric/patterns/
```

You can then use them like any other Patterns, but they won't be public unless you explicitly submit them as Pull Requests to the Fabric project. So don't worry—they're private to you.

This feature works with all openai and ollama models but does NOT work with claude. You can specify your model with the `-m` flag

Helper Apps

Fabric also makes use of some core helper apps (tools) to make it easier to integrate with your various workflows. Here are some examples:

to_pdf

`to_pdf` is a helper command that converts LaTeX files to PDF format. You can use it like this:

This will create a PDF file from the input LaTeX file in the same directory.

You can also use it with stdin which works perfectly with the `write_latex` pattern:

```
echo "ai security primer" | fabric --pattern write_latex | to_pdf
```

This will create a PDF file named `output.pdf` in the current directory.

to_pdf Installation

To install `to_pdf`, install it the same way as you install Fabric, just with a different repo name.

```
go install github.com/danielmiessler/fabric/plugins/tools/to_pdf@latest
```

Make sure you have a LaTeX distribution (like TeX Live or MiKTeX) installed on your system, as `to_pdf` requires `pdflatex` to be available in your system's PATH.

pbpaste

The [examples](#) use the macOS program `pbpaste` to paste content from the clipboard to pipe into `fabric` as the input. `pbpaste` is not available on Windows or Linux, but there are alternatives.

On Windows, you can use the PowerShell command `Get-Clipboard` from a PowerShell command prompt. If you like, you can also alias it to `pbpaste`. If you are using classic PowerShell, edit the file

`~\Documents\WindowsPowerShell\profile.ps1`, or if you are using PowerShell Core, edit `~\Documents\PowerShell\profile.ps1` and add the alias,

```
Set-Alias pbpaste Get-Clipboard
```

On Linux, you can use `xclip -selection clipboard -o` to paste from the clipboard. You will likely need to install `xclip` with your package manager. For Debian based systems including Ubuntu,

```
sudo apt update  
sudo apt install xclip -y
```

You can also create an alias by editing `~/.bashrc` or `~/.zshrc` and adding the alias,

```
alias pbpaste='xclip -selection clipboard -o'
```

Meta

Note

Special thanks to the following people for their inspiration and contributions!

- *Jonathan Dunn* for being the absolute MVP dev on the project, including spearheading the new Go version, as well as the GUI! All this while also being a full-time medical doctor!
- *Caleb Sima* for pushing me over the edge of whether to make this a public project or not.
- *Eugen Eisler* and *Frederick Ros* for their invaluable contributions to the Go version
- *Joel Parish* for super useful input on the project's Github directory structure..
- *Joseph Thacker* for the idea of a `-c` context flag that adds pre-created context in the `./config/fabric/` directory to all Pattern queries.
- *Jason Haddix* for the idea of a stitch (chained Pattern) to filter content using a local model before sending on to a cloud model, i.e., cleaning customer data using `llama2` before sending on to `gpt-4` for analysis.
- *Andre Guerra* for assisting with numerous components to make things simpler and more maintainable.

Primary contributors



